Data Analytics

BCS-052 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** theory answers are short and diagram-based. The **numericals (PCA, regression, Flajolet-Martin, windows, Apriori, K-Means)** are solved inside their own units, in the format: formula, then substitution, then boxed answer.

UNIT 1: DATA ANALYTICS LIFECYCLE & BIG DATA

Section A: 2-Mark Questions

Q1. Structured, semi-structured and unstructured data. *[2 Marks]*

Ans.

Structured	Semi-structured	Unstructured
Fixed schema (rows and columns)	Tags / keys, flexible schema	No format
RDBMS: bank records	XML, JSON, email	Images, videos, social posts (**about 80%** of all data)

Q2. The V's of Big Data; analysis vs. reporting. *[2 Marks]*

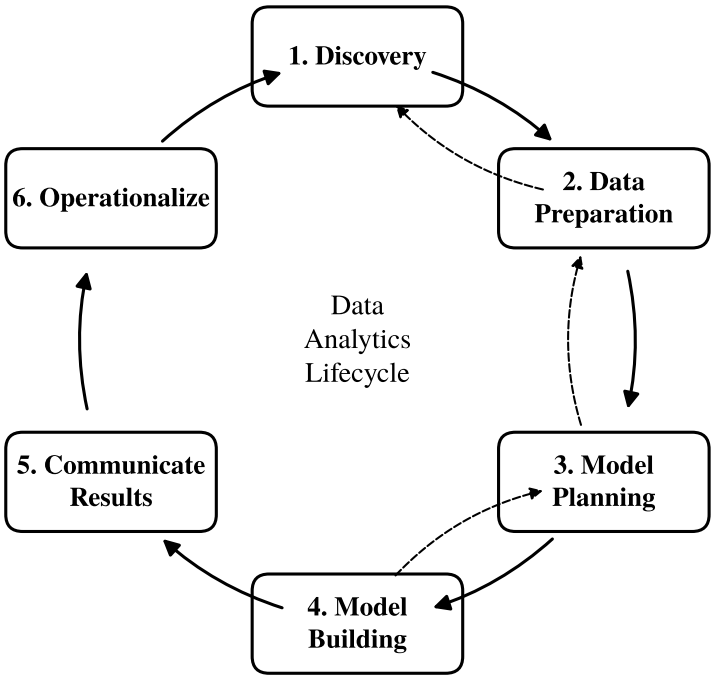
Ans.

- **V's: Volume** (size), **Velocity** (speed), **Variety** (types), **Veracity** (trustworthiness), **Value** (useful insight).
- **Reporting** shows **what happened** (organised summaries, dashboards). **Analysis** explains **why it happened and what will happen** (insights, predictions).

Section B: 7-Mark Questions

Q3. Explain the 6 phases of the Data Analytics Lifecycle with a diagram. *[7 Marks]*

Ans. The **Data Analytics Lifecycle** is an **iterative six-phase process** for data science projects. The team can **go back** to an earlier phase whenever needed.



Data Analytics Lifecycle: solid arrows show the normal flow, dashed arrows show going back to an earlier phase

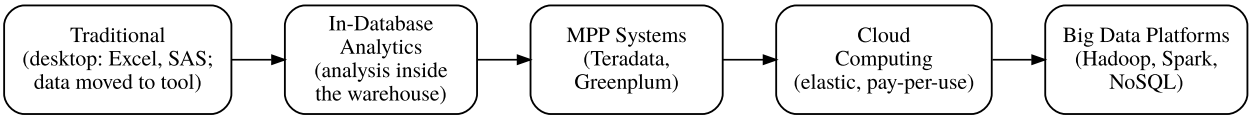
1. **Discovery:**
   - Understand the business problem, the stakeholders and the resources.
   - Form **initial hypotheses**.
   - *Exit question:* is there enough information to draft a plan?
2. **Data preparation:**
   - Set up an **analytic sandbox** and run **ETLT**.
   - Clean the data: missing values, outliers.
   - Usually **50 to 80% of the project time**.
   - Tools: Hadoop, OpenRefine.
3. **Model planning:**
   - Choose techniques and variables with **EDA**.
   - Tools: R, SQL, SAS.
4. **Model building:**
   - Split into train and test sets, build and tune the models.
   - Check that the model is robust.
   - Tools: R, Python, Spark MLlib.
5. **Communicate results:** compare with the success criteria and present the findings with visuals.
6. **Operationalize:** run a pilot, **deploy**, monitor and retrain. Deliver the reports and code.

Phase	Output
Discovery	Problem statement + hypotheses
Data preparation	Clean dataset
Model planning	Model plan
Model building	Working model
Communicate results	Findings
Operationalize	Deployed system

Conclusion: used in churn prediction, fraud detection and demand forecasting projects.

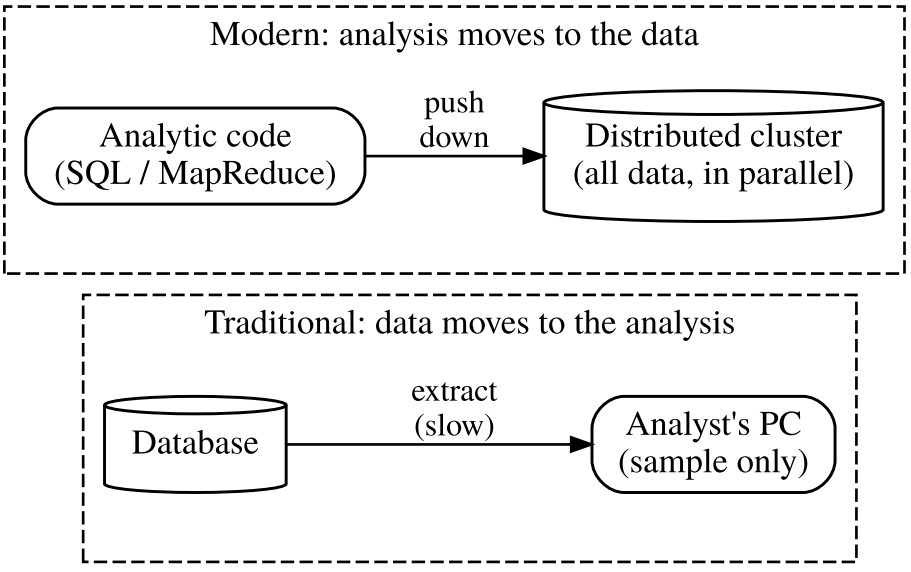
Q4. Explain the evolution of analytic scalability and the need for Big Data platforms. *[7 Marks]*

Ans. Analytic scalability is the ability to handle **growing data** by adding resources. Analytics has moved from desktop tools to **distributed platforms**.



Evolution of analytic scalability

1. **Traditional:** data is **extracted to the analyst's PC** (SAS, Excel). This is slow and only **samples** can be analysed.
2. **In-database analytics:** the analysis runs **inside the warehouse**, so there is no data movement.
3. **MPP** (Teradata, Greenplum): data is partitioned across nodes and processed **in parallel**.
4. **Cloud: elastic, pay-per-use** resources (AWS EMR, BigQuery).
5. **Big Data platforms** (Hadoop, Spark): **distributed storage and processing on commodity hardware**, with data locality.



Old approach vs. modern approach: move the analysis to the data

Need for Big Data platforms:

- **Volume:** petabytes.
- **Variety:** unstructured data.
- **Velocity:** real-time streams.
- **Cost:** cheap horizontal scaling.
- **Fault tolerance:** replication.
- **ML on the full data**, not just samples.

Traditional	Big Data platform
Vertical scaling	**Horizontal** scaling
Structured data only	All types
Schema-on-write	**Schema-on-read**

UNIT 2: ADVANCED DATA ANALYSIS TECHNIQUES

Section A: 2-Mark Questions

Q1. What is PCA? Define a Bayesian Network. Core idea of Fuzzy Logic. *[2 Marks]*

Ans.

- **PCA:** reduces dimensions by converting correlated variables into **uncorrelated principal components** (the eigenvectors of the covariance matrix), ordered by variance (eigenvalue).
- **Bayesian network:** a **DAG** whose nodes are random variables, with **conditional probability tables**: $P(X_1..X_n) = \prod P(X_i \mid \text{Parents}(X_i))$.
- **Fuzzy logic:** truth is a **degree from 0 to 1** (membership value μ), not just true or false. Example: 30°C is "Hot" with μ = 0.7.

Section B: 7-Mark Questions

Q2. Explain regression modelling and multivariate analysis. *[7 Marks]*

Ans. Regression estimates how a **dependent variable Y depends on predictors X**, so that Y can be predicted. **Multivariate analysis** studies **many variables together**.



Types of regression and multivariate techniques

1. **Simple linear regression:** $\widehat{Y} = b_0 + b_1X$, with $b_1 = \dfrac{\sum (X - \bar{X})(Y - \bar{Y})}{\sum (X - \bar{X})^2}$ and $b_0 = \bar{Y} - b_1\bar{X}$.
2. **Multiple regression:** $\widehat{Y} = b_0 + b_1X_1 + \ldots + b_nX_n$, with $b = (X^TX)^{-1}X^TY$.

3. **Logistic regression:** $P(Y=1) = \frac{1}{1+e^{-(b_0+b_1X)}}$, for a binary Y.
4. **Assumptions:** linearity, independent errors, homoscedasticity, normal residuals, no multicollinearity.
5. **Fit:** $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$, plus RMSE and p-values.
6. **Multivariate techniques:** multiple regression, **PCA / factor analysis**, **cluster analysis**, discriminant analysis, **MANOVA**.

Univariate	Bivariate	Multivariate
1 variable	2 variables	3 or more variables
Mean, histogram	Correlation	Regression, PCA

Q3. Numerical: PCA. (a) Eigenvalues 4.5, 2.8, 1.2, 0.4, 0.1. (b) 2-D points. *[7 Marks]*

Ans. (a) $\sum \lambda = 9.0$, and the variance of each component is $\frac{\lambda_k}{9.0} \times 100$.

PC	1	2	3	4	5
% variance	50.0	31.1	13.3	4.4	1.1
Cumulative %	50.0	81.1	**94.4**	98.9	100

3 components are needed for $\geq 90\%$ variance

(b) Points (2,1), (3,5), (4,3), (5,6), (6,7), (7,8)

1. **Mean:** $\bar{x} = 4.5$, $\bar{y} = 5$.
2. **Sums of deviations:** $\sum (x-\bar{x})^2 = 17.5$, $\sum (y-\bar{y})^2 = 34$, $\sum (x-\bar{x})(y-\bar{y}) = 22$.
3. **Covariance matrix** (dividing by n = 6):

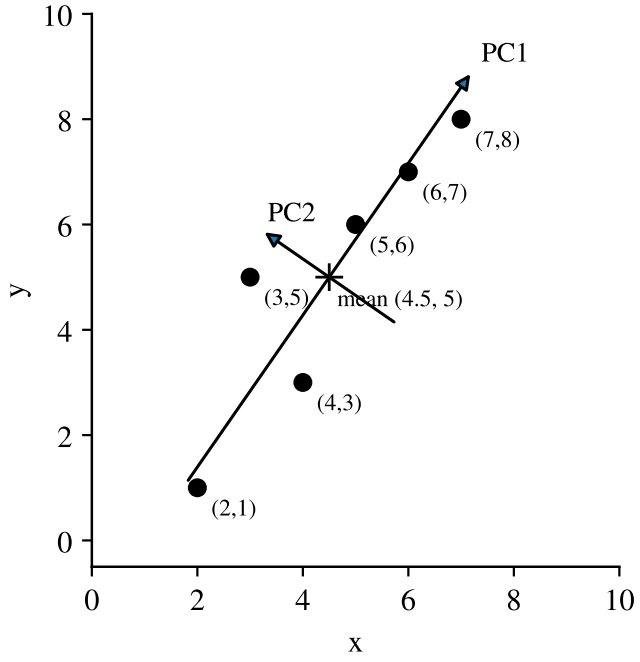
$$\text{Cov} = \begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix}$$

4. **Eigenvalues:**

$$\lambda^2 - 8.58\lambda + 3.08 = 0 \Rightarrow \lambda_1 = 8.21,\ \lambda_2 = 0.38$$

5. **Eigenvector for λ_1:** $(2.92 - 8.21)e_1 + 3.67e_2 = 0$, so $e_2 = 1.44e_1$.

$$PC_1 = (0.57,\ 0.82) \text{ explains } \frac{8.21}{8.59} = 95.6\% \text{ of the variance}$$



Data points with the first principal component direction

Q4. Numerical: regression (hours vs. scores) and standard deviation. *[7 Marks]*

Ans. (a) X = 2, 4, 6, 8, 10 and Y = 50, 55, 65, 70, 78.

- $\bar{X} = 6$, $\bar{Y} = 63.6$.
- $\sum \left(X - \bar{X}\right)\left(Y - \bar{Y}\right) = 54.4 + 17.2 + 0 + 12.8 + 57.6 = 142$ and $\sum \left(X - \bar{X}\right)^2 = 40$.

$$b_1 = \frac{142}{40} = 3.55, \qquad b_0 = 63.6 - 3.55\,(6) = 42.3$$

$$\widehat{Y} = 42.3 + 3.55X, \qquad \widehat{Y}(7) = 42.3 + 24.85 = 67.15$$

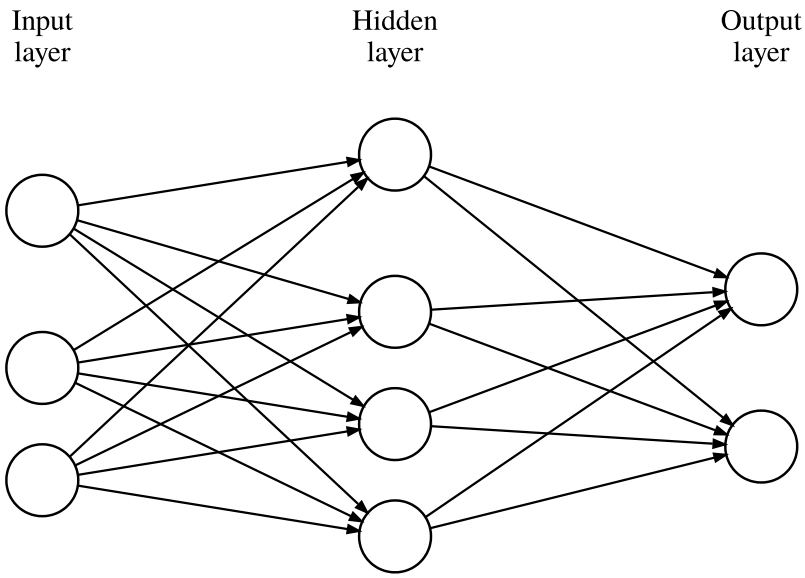
(b) Data {12, 15, 20, 22, 25, 30}:

$$\bar{x} = \frac{124}{6} = 20.67, \quad \sum (x - \bar{x})^2 = 215.33, \quad \sigma = \sqrt{\frac{215.33}{6}} = 5.99 \ (\text{sample } s = 6.56)$$

- **Comment:** CV = 5.99 / 20.67 = **29%**, so the data has a **moderate spread**. It is fairly symmetric (the median 21 is close to the mean) and has no outliers.

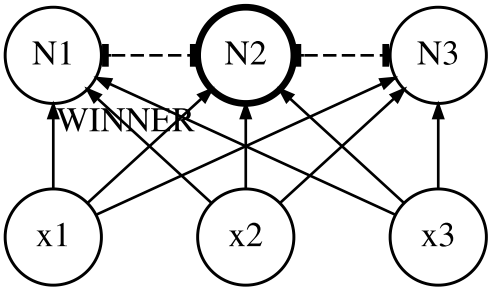
Q5. Neural networks: competitive learning and generalization. *[7 Marks]*

Ans. An **ANN** has **input, hidden and output layers** of neurons, each computing $y = f\left(\sum w_i x_i + b\right)$.



Feed-forward neural network architecture

1. **Architectures:** feed-forward (MLP), recurrent (RNN), competitive (SOM).
2. **Competitive learning (unsupervised):**
   - Output neurons **compete**. The **winner** is the one closest to the input: $j^* = \arg\min_j \| x - w_j \|$.
   - **Only the winner learns:** $w_{j^*} \leftarrow w_{j^*} + \eta \left(x - w_{j^*} \right)$.
   - The result is **cluster prototypes**. In a **Kohonen SOM**, the neighbours learn too.
3. **Generalization:** the network performs well on **unseen data**.
   - **Overfitting** is the main threat.
   - Improve it with **cross-validation, early stopping, regularization, dropout and more data**.

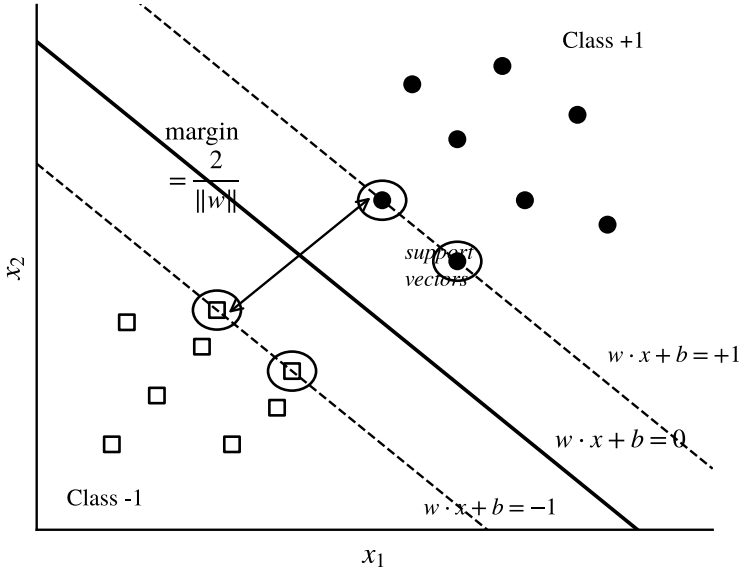


Competitive (winner-take-all) learning layer

Competitive learning	Backpropagation
Unsupervised	Supervised
Only the winner is updated	All weights are updated
Produces clusters	Produces an input-output mapping

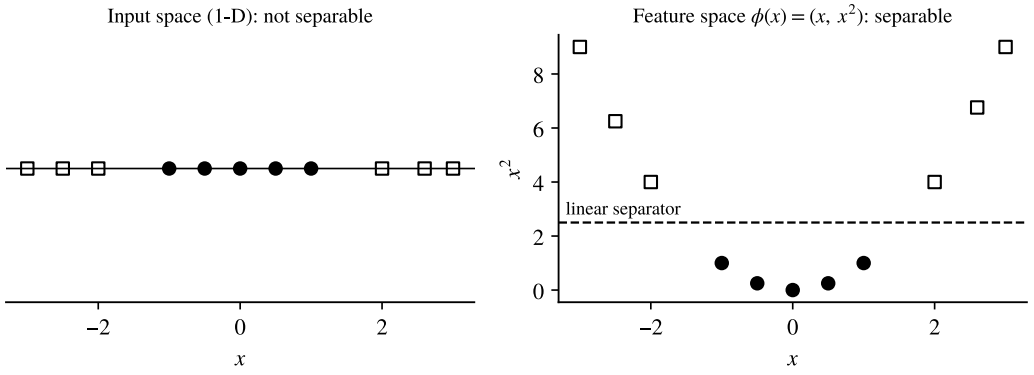
Q6. Explain SVM and kernel methods. *[7 Marks]*

Ans. An **SVM** finds the **maximum-margin hyperplane** $w \cdot x + b = 0$. **Kernels** handle non-linear data by implicitly mapping it to a higher dimension.



Maximum-margin hyperplane with support vectors

1. **Margin** $= 2/\|w\|$. The optimization is $\min \frac{1}{2}\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$.
2. **Support vectors:** the points on the margin. Only they decide the hyperplane.
3. **Soft margin:** slack variables ξ with penalty C.
4. **Kernel trick:** $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$.
    - Kernels: linear, polynomial $(x \cdot y + c)^d$, **RBF** $e^{-\gamma\|x-y\|^2}$.



Kernel trick: data not separable in 1-D becomes separable after mapping x to (x, x²)

Conclusion: used for text, image and gene classification.

UNIT 3: MINING DATA STREAMS

Section A: 2-Mark Questions

Q1. What is a data stream? Decaying vs. sliding window. *[2 Marks]*

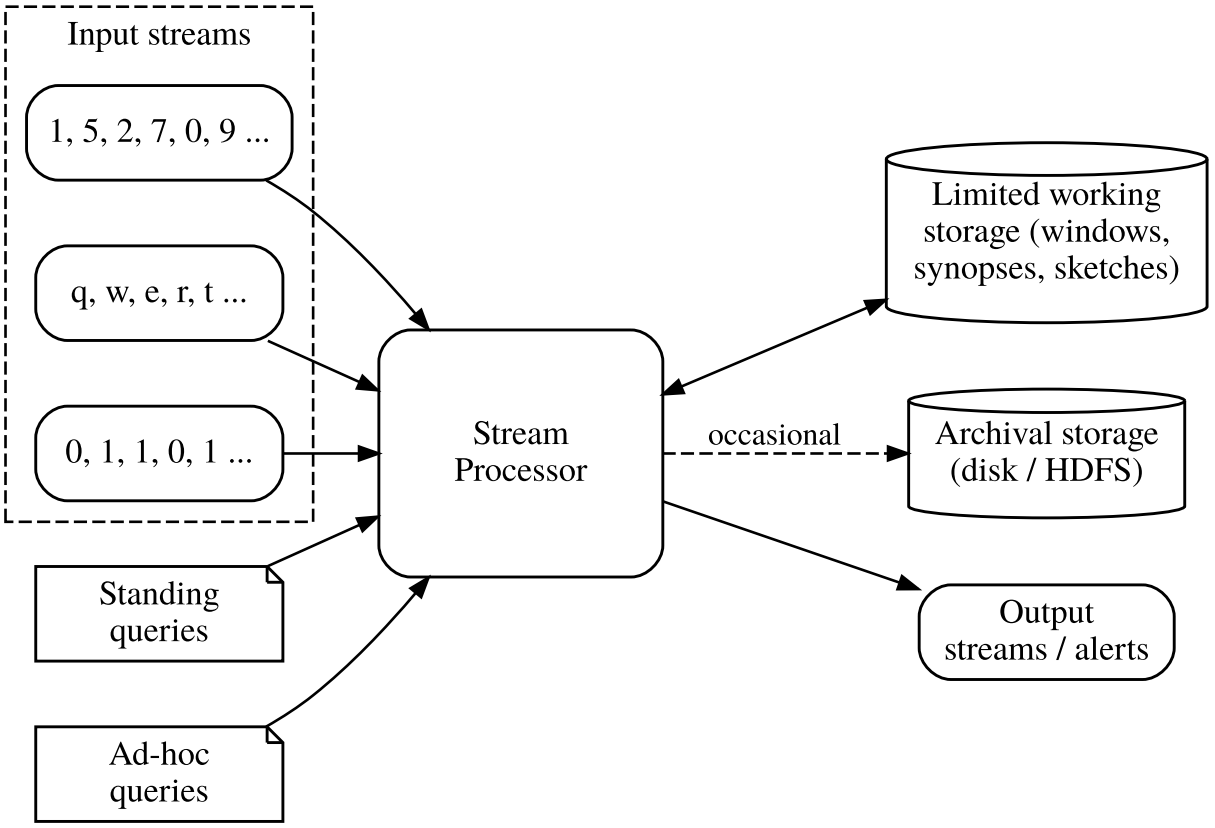
Ans. A **data stream** is a **continuous, unbounded, fast, time-ordered** sequence of data. It is read **once**, with **limited memory**. Examples: sensors, tweets, stock ticks.

Sliding window	Decaying window
Only the **last N** elements count	**All** elements count, with weight $(1-c)^{\text{age}}$
Old data is dropped completely	Old data **fades** gradually
Stores N elements	Stores **one running sum**: $S = S(1-c) + a_t$

Section B: 7-Mark Questions

Q2. Explain the architecture of a stream computing model. *[7 Marks]*

Ans. Stream computing processes **data in motion** as it arrives, using a **DSMS (Data Stream Management System)**.



Architecture of a data stream management system

1. **Input streams:** many sources, each with its own rate.
2. **Stream processor:** reads each element **once** and applies the queries.

3. **Working storage (limited memory):** holds **windows, samples and sketches** (Bloom filter, FM sketch).
4. **Archival storage:** large and slow. It is **not used for real-time answers**.
5. **Standing queries** (continuous, for example "alert if temperature > 50") and **ad-hoc queries** (one-time).
6. **Output streams:** alerts and dashboards.

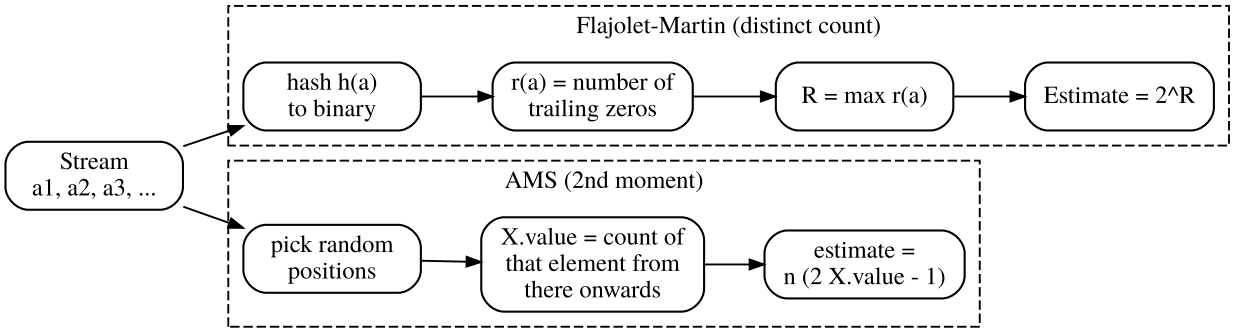
- **Key algorithms:** sampling, Bloom filter, **Flajolet-Martin**, **AMS**, DGIM.
- **Tools:** Kafka, Storm, Flink, Spark Streaming.

DBMS	DSMS
Finite, stored data	**Infinite** stream
Many passes	**One pass**
One-time queries	**Continuous** queries
Exact answers	**Approximate** answers

Q3. Estimating moments and counting distinct elements (Flajolet-Martin and AMS). *[7 Marks]*

Ans. The **k-th moment** of a stream is $F_k = \sum_i m_i^k$:

- F_0 = the number of **distinct** elements.
- F_1 = the stream length.
- F_2 = the "surprise number".



Flajolet-Martin and AMS: one pass, small memory

Flajolet-Martin (F_0):

1. Hash each element to binary.
2. r = the number of **trailing zeros**.
3. R = the maximum r.
4. **Estimate =** 2^R**.**

Why it works: the probability of seeing r trailing zeros is 2^{-r}. Use many hash functions and take the **median of the averages**.

AMS (F_2): pick random positions. X.value = the count of that element from that position onwards. Then:

$$F_2 \approx n\,(2X.\text{value} - 1)$$

- Example: stream a b c b d a c d a b d c a a b (n = 15). The true $F_2 = 25 + 16 + 9 + 9 = 59$.
- X at position 3 (element c) counts 3 occurrences, so the estimate is $15\,(2 \times 3 - 1) = 75$. Averaging many such estimates gets close to 59.

Q4. Numerical: Flajolet-Martin, decaying window, sliding window. *[7 Marks]*

Ans. (a) S = {1, 3, 2, 1, 2, 3, 4, 3, 1, 2, 3, 1}, with $h(x) = (6x + 1) \mod 5$.

x	1	2	3	4
h(x)	7 mod 5 = 2	13 mod 5 = 3	19 mod 5 = 4	25 mod 5 = 0
Binary	010	011	**100**	000
Trailing zeros r	1	0	**2**	0 (taken as 0)

$$R = 2 \Rightarrow \text{distinct} \approx 2^2 = 4 \quad (\text{actual} = \{1, 2, 3, 4\} = 4\ \checkmark)$$

(b) Decaying window:

$$120,000 \times 0.35 = 42,000 \text{ records per minute retained}$$

- **Justification:** the stream is infinite, so not everything can be stored. A decaying window keeps **recent data important**, needs **constant memory** (one running sum), and still tracks trends.

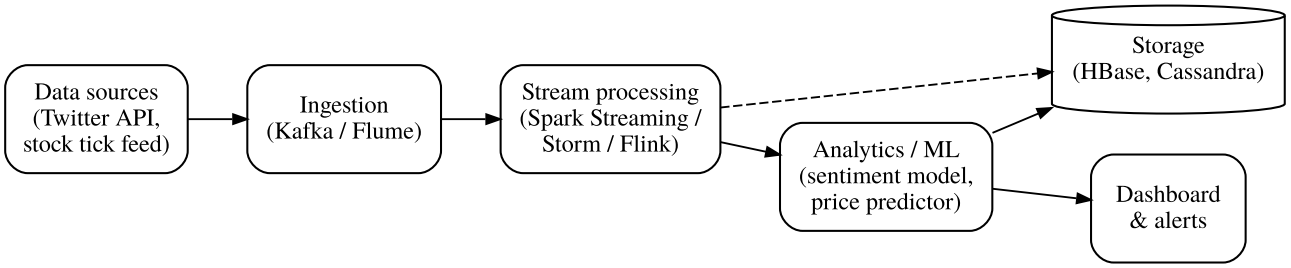
(c) Sliding window (size 200, slide 50, 1000 elements):

$$\frac{N - W}{S} + 1 = \frac{1000 - 200}{50} + 1 = 17 \text{ windows}$$

- **Latency:** a new result comes every 50 elements, so results are **fresher (low latency)**. But the windows overlap by 150 elements, so each element is processed up to 4 times (**more computation**).

Q5. RTAP applications: sentiment analysis and stock prediction case study. *[7 Marks]*

Ans. A **Real-Time Analytics Platform (RTAP) ingests, processes and analyses streams within milliseconds**, so decisions can be made immediately.



Real-time analytics pipeline

Layers:

1. Sources.
2. **Ingestion:** Kafka, Flume.
3. **Processing:** Spark Streaming, Storm.
4. **ML / analytics.**
5. **Storage:** HBase, Cassandra.

6. **Dashboard and alerts.**

Case study 1: Twitter sentiment analysis

1. Stream tweets for a brand into **Kafka**.
2. **Clean** them: remove URLs and stop-words, then tokenize.
3. **Classify** each tweet as positive, negative or neutral (VADER, Naive Bayes, LSTM).
4. Aggregate in a **sliding window** (% positive in the last 5 minutes).
5. **Alert** the PR team if negative sentiment spikes.

Case study 2: Stock prediction

- Ingest price ticks and news.
- Compute moving averages, RSI and sentiment over windows.
- Predict with ARIMA or LSTM.
- Generate **buy/sell signals in milliseconds**.

Other uses: fraud detection, IoT monitoring, Uber surge pricing.

Batch analytics	Real-time analytics
Stored data, hours of latency	Live data, **milliseconds**
Hadoop, Hive	Kafka, Spark Streaming

UNIT 4: FREQUENT ITEMSETS & CLUSTERING

Section A: 2-Mark Questions

Q1. Apriori property; K-Means vs. Hierarchical clustering. *[2 Marks]*

Ans. Apriori property: *every subset of a frequent itemset is frequent*. Equivalently, **if an itemset is infrequent, all its supersets are infrequent**. This lets the algorithm prune candidates.

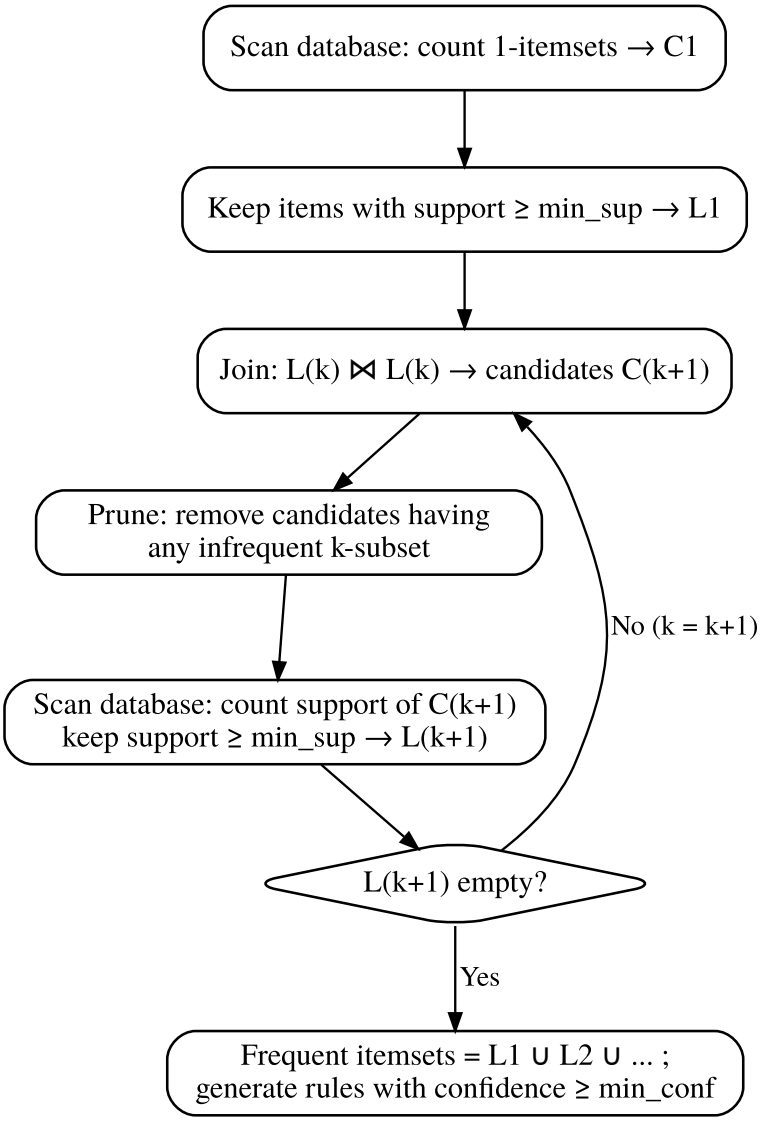
K-Means	Hierarchical
K must be given	No K needed (dendrogram)
Fast, $O(nkt)$	Slow, $O(n^2)$
Depends on the initial centroids	Deterministic

Section B: 7-Mark Questions

Q2. Trace the Apriori algorithm. *[7 Marks]*

Ans. Apriori finds frequent itemsets **level by level**: join L_k to get C_{k+1}, **prune** candidates that have an infrequent subset, count the support, and keep those with support ≥ min_sup.

$$\text{Support}(X) = \frac{\text{count}(X)}{N}, \qquad \text{Confidence}(X \Rightarrow Y) = \frac{\text{count}(X \cup Y)}{\text{count}(X)}$$



Apriori algorithm

(a) Transactions T1 {A,B,C}, T2 {A,B}, T3 {A,C}, T4 {B,C}, T5 {A,B,C}. Minimum support = 60% (count ≥ 3).

Itemset	A	B	C	AB	AC	BC	ABC
Count	4	4	4	3	3	3	**2**
Frequent?	✓	✓	✓	✓	✓	✓	×

- {A,B,C} **passes the subset check** (all its pairs are frequent) but **fails the support check** (2 < 3), so it is pruned.

$$\text{Frequent: } \{A\}, \{B\}, \{C\}, \{A, B\}, \{A, C\}, \{B, C\}$$

(b) Grocery data:

- T10 {Beer, Nuts, Diaper}
- T20 {Beer, Coffee, Diaper}
- T30 {Beer, Diaper, Eggs}
- T40 {Nuts, Eggs, Milk}
- T50 {Nuts, Coffee, Diaper, Eggs, Milk}

Support 50% means count ≥ 3; confidence threshold 60%.

Item	Beer	Nuts	Diaper	Coffee	Eggs	Milk
Count	3	3	4	2 ×	3	2 ×

- Pairs: {Beer, Diaper} = **3** ✓. All the other pairs are ≤ 2.

Rule	Support	Confidence	Strong?
Beer ⇒ Diaper	60%	3/3 = **100%**	✓
Diaper ⇒ Beer	60%	3/4 = **75%**	✓

(The same method applies if your paper has a 6th transaction: just recount.)

Q3. Numerical: K-Means with Manhattan distance (8 points, K = 3). *[7 Marks]*

Ans. Points: A1(2,10), A2(2,5), A3(8,4), A4(5,8), A5(7,5), A6(6,4), A7(1,2), A8(4,9). Initial centres C1 = A1, C2 = A4, C3 = A7. Distance: $d = |x_2 - x_1| + |y_2 - y_1|$.

Iteration 1

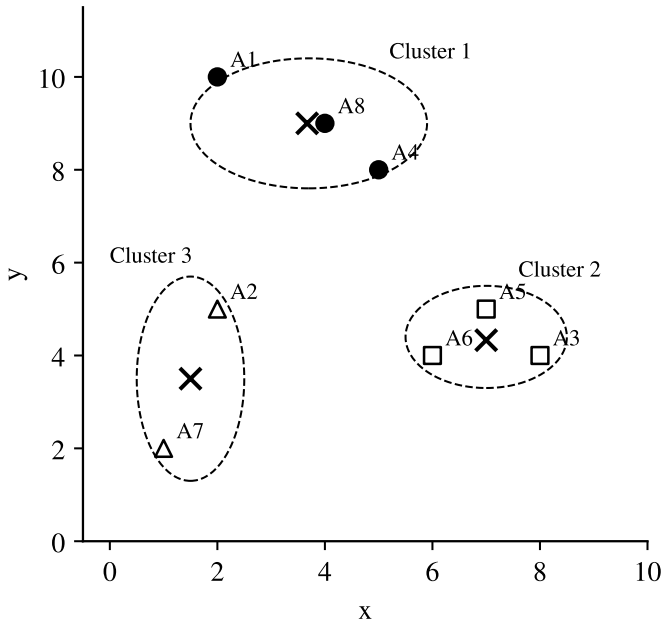
Point	C1 (2,10)	C2 (5,8)	C3 (1,2)	Cluster
A1	0	5	9	C1
A2	5	6	**4**	C3
A3	12	**7**	9	C2
A4	5	**0**	10	C2
A5	10	**5**	9	C2
A6	10	**5**	7	C2
A7	9	10	**0**	C3
A8	3	**2**	10	C2

New centres after iteration 1:

$$C_1 = (2, 10), \quad C_2 = \left(\frac{8+5+7+6+4}{5}, \frac{4+8+5+4+9}{5}\right) = (6, 6), \quad C_3 = (1.5, 3.5)$$

Later iterations:

- Iteration 2: C1 = {A1, A8}, C2 = {A3, A4, A5, A6}, giving centres (3, 9.5) and (6.5, 5.25).
- Iteration 3: C1 = {A1, A4, A8}, C2 = {A3, A5, A6}.
- Iteration 4: **no change**, so the algorithm has converged.

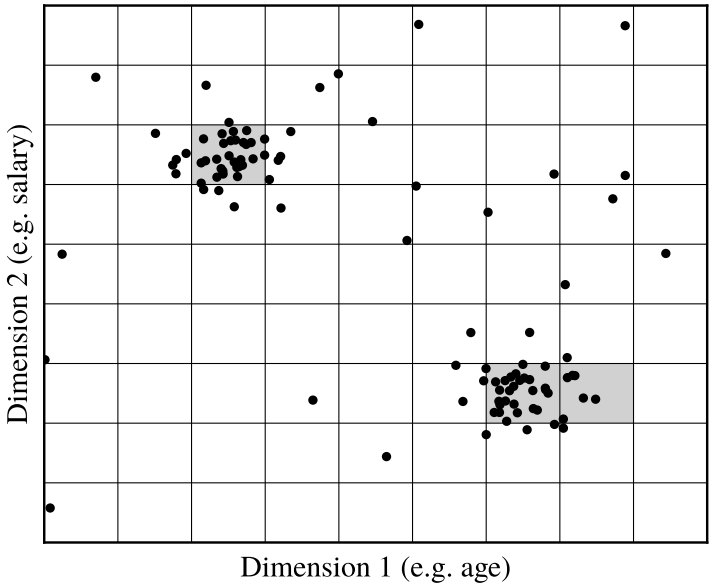


Final three clusters (x = final centre)

$$C_1 = (3.67, 9), \quad C_2 = (7, 4.33), \quad C_3 = (1.5, 3.5)$$

Q4. Explain the CLIQUE and PROCLUS algorithms. *[7 Marks]*

Ans. High-dimensional data suffers from the **curse of dimensionality**: clusters exist only in **subspaces**. **CLIQUE** is a **grid and density, bottom-up** method. **PROCLUS** is a **k-medoid, top-down projected** method.



CLIQUE: dense grid units (shaded) in 2-D; dense units in both 1-D projections suggest candidate dense cells

CLIQUE:

1. Split each dimension into ξ **intervals**, forming a grid.
2. A unit is **dense** if its share of points is greater than τ.
3. Use an **Apriori-like** bottom-up search: if a k-D unit is dense, all its (k−1)-D projections are dense.
4. A cluster = **connected dense units**.

5. Describe each cluster as a **DNF rule**, for example $(30 \le \text{age} < 50) \wedge (4 \le \text{salary} < 8)$.

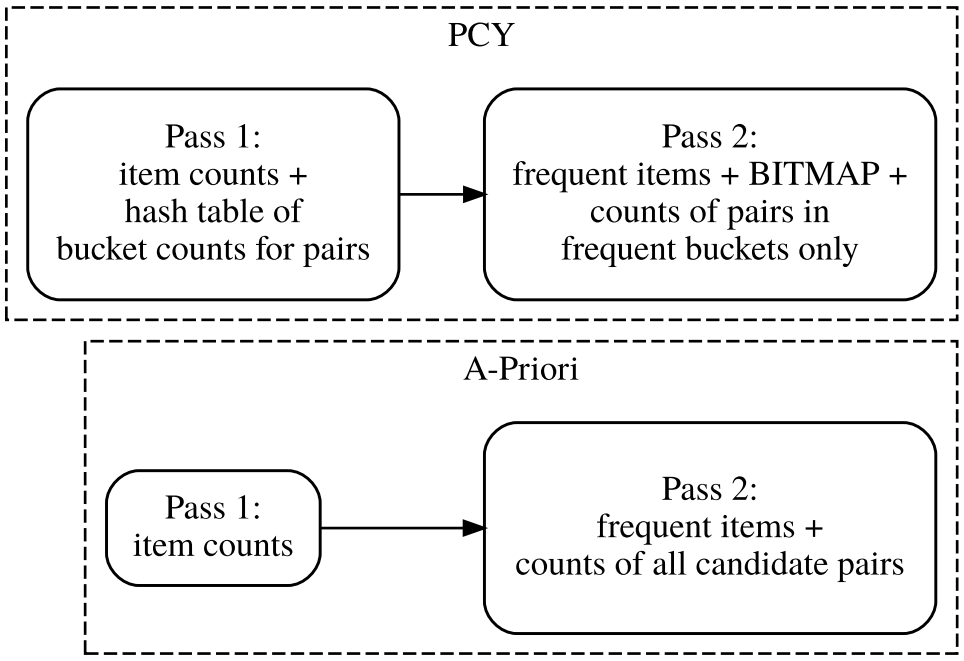
PROCLUS:

1. Choose **k medoids** from a sample, greedily, so that they are well separated.
2. For each medoid, find the **relevant dimensions** (those where its locality is closest).
3. Assign each point using the **Manhattan segmental distance** over those dimensions.
4. Replace bad medoids and repeat.
5. Refine, and mark **outliers**.

CLIQUE	PROCLUS
Bottom-up, grid-based	Top-down, medoid-based
Inputs: ξ, τ	Inputs: k, l
Clusters may overlap	Disjoint clusters
Arbitrary shapes	Spherical in their subspace

Q5. Handling large datasets in main memory: limited-pass algorithms. *[7 Marks]*

Ans. When baskets do not fit in memory, the **second pass (counting pairs)** is the bottleneck. **Limited-pass** algorithms reduce the passes and the memory needed.



Main-memory use in A-Priori vs. PCY

1. **PCY:**
   - In pass 1, also **hash every pair into buckets**. Turn the counts into a **bitmap**.
   - In pass 2, count a pair only if **both items are frequent and its bucket is frequent**.
2. **Multistage:** a second hash table in an extra pass, which gives fewer false positives.
3. **Multihash:** several hash tables in the same pass.

4. **Random sampling:** run Apriori **in memory on a sample** with a lowered threshold. False positives and negatives are possible.
5. **SON (2 passes):** find the frequent itemsets **per chunk**, then count those candidates over the full data. **No false negatives**, and it works with **MapReduce**.
6. **Toivonen:** use a sample plus the **negative border**. If no border itemset is frequent, the answer is exact. Otherwise, repeat with a new sample.

Algorithm	Passes	Exact?
PCY	2	Yes
SON	2	Yes
Sampling	1	No
Toivonen	1 to 2	Yes (if it succeeds)

UNIT 5: FRAMEWORKS, HADOOP & R

Section A: 2-Mark Questions

Q1. Sharding; NoSQL; data types in R. *[2 Marks]*

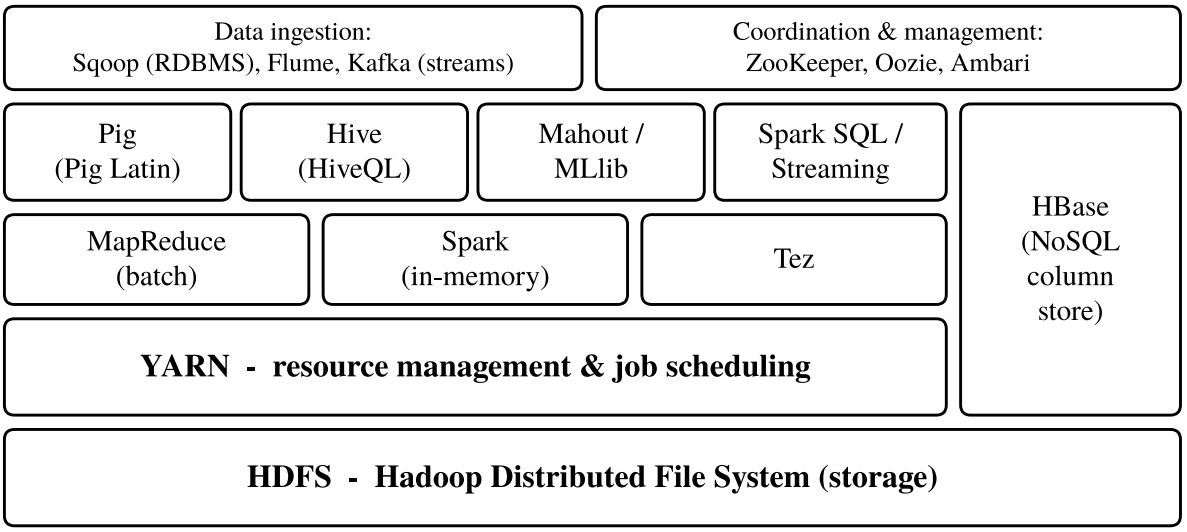
Ans.

- **Sharding: horizontal partitioning** of the rows across servers using a **shard key**. It gives horizontal scaling (MongoDB, Cassandra).
- **NoSQL:** a non-relational, **schema-flexible, horizontally scalable** database that follows **BASE**. Types: key-value (Redis), document (MongoDB), column (HBase), graph (Neo4j).
- **R data types:**
  - numeric `12.5`, integer `12L`, character `"AKTU"`, logical `TRUE`, complex `3+2i`, raw.
  - Structures: vector, matrix, list, factor, **data frame**.

Section B: 7-Mark Questions

Q2. Explain the Hadoop ecosystem and the HDFS architecture. *[7 Marks]*

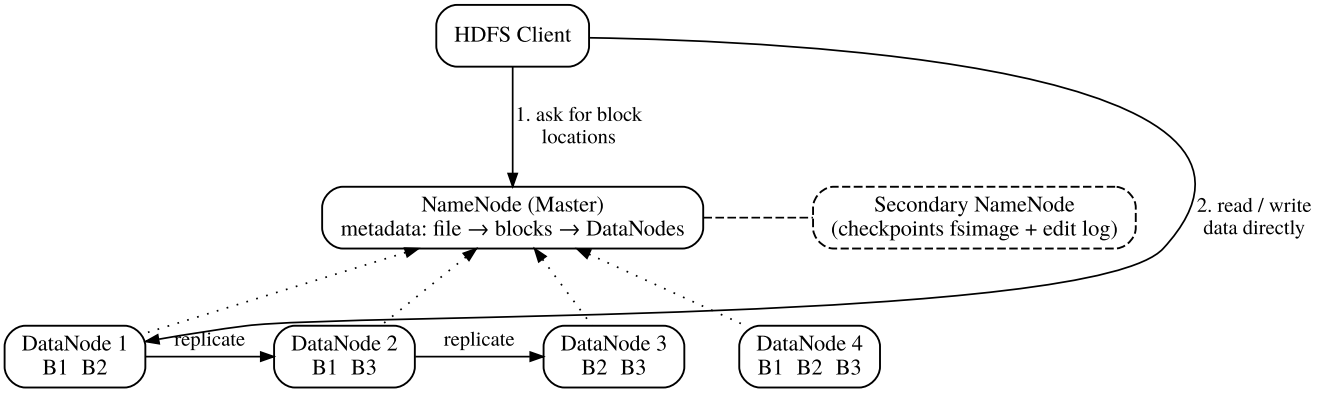
Ans. Hadoop is an open-source framework for the **distributed storage (HDFS) and processing (MapReduce / YARN)** of big data on commodity hardware.



Hadoop ecosystem (layered view)

Ecosystem:

- **HDFS** (storage), **YARN** (resources), **MapReduce / Spark** (processing).
- **Hive** (SQL), **Pig** (scripts), **HBase** (NoSQL).
- **Sqoop / Flume / Kafka** (ingestion), **ZooKeeper / Oozie** (coordination).



HDFS architecture (block size 128 MB, replication factor 3)

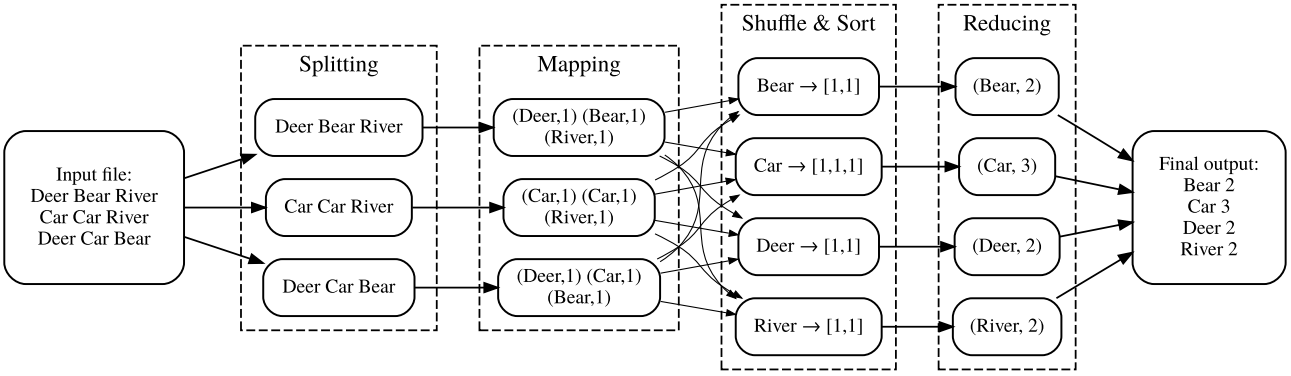
HDFS architecture:

1. **NameNode (master):** holds the **metadata** (file → blocks → DataNodes). It does not store data.
2. **DataNodes (slaves):** store the **blocks** and send **heartbeats** every 3 seconds.
3. **Secondary NameNode: checkpoints** (merges the fsimage and the edit log). It is not a backup.
4. **Blocks: 128 MB** each, with **replication factor 3** and rack awareness.
5. **Write:** the client asks the NameNode, then writes to the DataNodes in a **pipeline**. **Read:** the client gets the locations, then reads from the nearest DataNode.

HDFS	Normal file system
Distributed	Single machine
128 MB blocks	4 KB blocks
Replication for fault tolerance	RAID / backups
Write once, read many	Random read and write

Q3. Explain the MapReduce framework with a diagram. *[7 Marks]*

Ans. MapReduce processes big data **in parallel**. **Map** turns input into **(key, value)** pairs, and **Reduce aggregates the values for each key**.



MapReduce data flow for Word Count

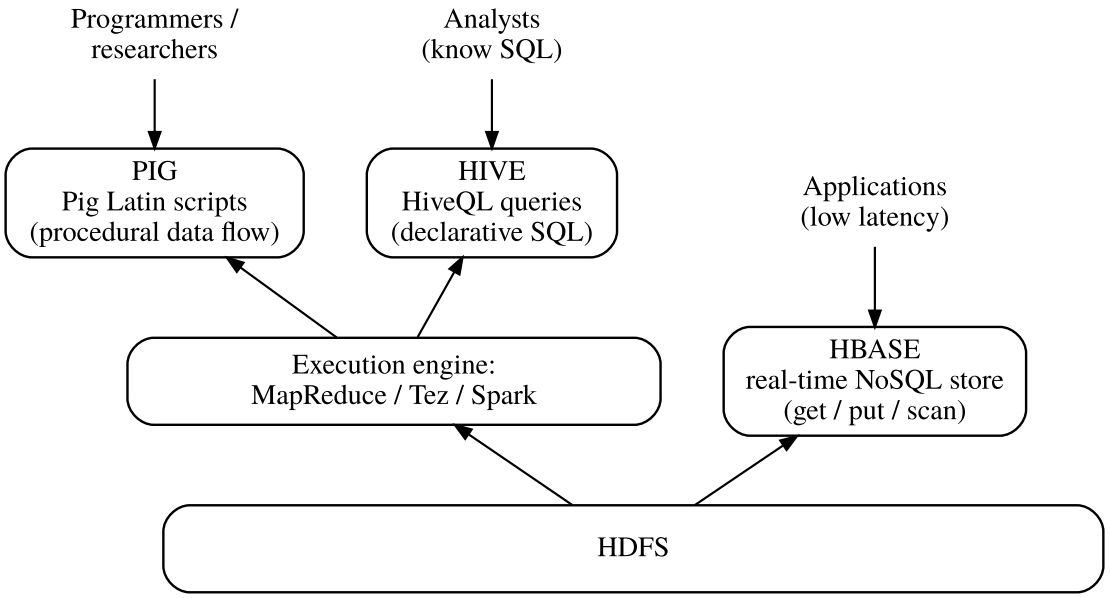
1. **Splitting:** the input is divided into splits (one per 128 MB block).
2. **Map:** `map(k1, v1) → list(k2, v2)`. For word count, emit (word, 1).
3. **Combiner** (optional): local aggregation, which reduces network traffic.
4. **Partitioner:** `hash(key) mod R` chooses the reducer.
5. **Shuffle and sort:** group the values by key, for example (Car, [1, 1, 1]).
6. **Reduce:** `reduce(k2, list(v2))`, giving (Car, 3).
7. **Output** is written to HDFS.

```
map(line):      for each word w: emit(w, 1)
reduce(w, L):   emit(w, sum(L))
```

Map	Reduce
Filter and transform	Aggregate
Many tasks	Fewer tasks
Output on local disk	Output in HDFS

Q4. Differentiate between Pig, Hive and HBase. *[7 Marks]*

Ans.



Where Pig, Hive and HBase sit in Hadoop

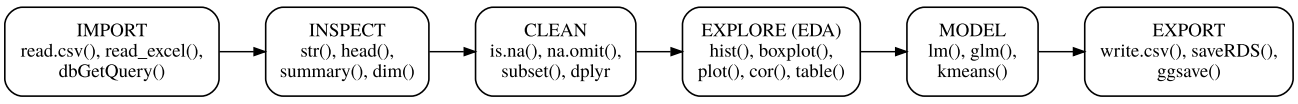
Basis	Pig	Hive	HBase
Type	Data-flow scripting	Data warehouse (SQL)	**NoSQL** column store
Language	**Pig Latin** (procedural)	**HiveQL** (declarative)	get / put / scan API
Processing	Batch	Batch	**Real-time**
Latency	High	High	**Milliseconds**
Schema	Optional	Schema-on-read	Flexible columns
Updates	No	Limited	**Row-level**
Best for	**ETL**	**Reports, BI**	**Online apps**
Developed by	Yahoo	Facebook	Based on Google BigTable

- **Pig example:** `grp = GROUP logs BY user; c = FOREACH grp GENERATE group, COUNT(logs);`
- **Hive example:** `SELECT region, SUM(amount) FROM sales GROUP BY region;`

Conclusion: in a typical pipeline, Pig cleans the data, Hive reports on it, and HBase serves it in real time.

Q5. Data import/export and EDA using R. *[7 Marks]*

Ans. R is a statistical language. **EDA (Exploratory Data Analysis)** summarizes and visualizes data before modelling.



Data analysis workflow in R

```r
df <- read.csv("students.csv")                  # import CSV
library(readxl); x <- read_excel("marks.xlsx")  # import Excel
write.csv(df, "clean.csv", row.names = FALSE)  # export
str(df); head(df); summary(df); dim(df)        # structure and summary
colSums(is.na(df))                             # missing values
mean(df$marks); sd(df$marks); table(df$branch) # statistics
cor(df[, 2:4])                                 # correlation
hist(df$marks); boxplot(marks ~ branch, df)    # plots
plot(df$hours, df$marks)                       # scatter plot
```

EDA steps:

1. Understand the structure.
2. Univariate statistics and plots.
3. Bivariate analysis and correlation.
4. Find missing values and outliers.
5. Form hypotheses.

Conclusion: R is widely used for statistics, research and finance analytics.